



Dokumentace k projektu

Implementace překladače imperativního jazyka IFJ25

Tým xlostap00, varianta TRP-izp

Vedoucí týmu:

Patrik Lošťák (xlostap00) 31%

Členové týmu:

Šimon Čorej (xcorejs00) 23%

Petr David Lanča (xlancap00) 23%

Sebastián Kuchta (xkuchts00) 23%

Rozšíření:

OPERATORS

BOOLTHEN

FUNEXP

26. listopadu 2025

Obsah

1	Úvod	2
2	Návrh a implementace	2
2.1	Lexikální analýza	2
2.2	Syntaktická analýza	2
2.2.1	Precedenční analýza ke zpracování výrazů	4
2.3	Sémantická analýza	4
2.4	Generátor cílového kódu	5
3	Použité datové struktury a algoritmy	6
3.1	Tabulka symbolů	6
3.2	Zásobníky	6
3.3	Abstraktní syntaktický strom (AST)	6
4	Organizace týmu a práce	7
4.1	Rozdělení práce mezi členy	7
5	Závěr	8
	Příloha A: Diagram konečného automatu	10
	Příloha B: LL Gramatika	11
	Příloha C: LL a Precedenční tabulka	12

1 Úvod

Cílem projektu bylo vytvořit program v programovacím jazyce C, který přeloží kód ve zdrojovém jazyce IFJ25 do cílového jazyka IFJcode25 (tzv. mezikódu). Zdrojový jazyk IFJ25 [2] je zjednodušená podmnožina jazyka Wren [3], což je jednoduchý, skriptovací, dynamicky typovaný a objektově orientovaný jazyk.

Samotný program funguje jako jednoduchá konzolová aplikace, která načte řídicí program v jazyce IFJ25 ze standardního vstupu a vygeneruje výsledný kód v jazyce IFJcode25 na standardní výstup; všechny ostatní výpisy, varování a chybová hlášení jsou posílána na standardní chybový výstup.

2 Návrh a implementace

Projekt jsme sestavili z několika námi implementovaných dílčích částí, které jsou představeny v této kapitole. V každé podkapitole jsou jednotlivé části detailněji popsány, popřípadě vysvětleno, jak spolu jednotlivé části spolupracují.

2.1 Lexikální analýza

Při tvorbě překladače jsme začali implementací lexikální analýzy. Pro lexikální analyzátor (scanner) byl nejdříve navržen konečný automat 5, který popisuje chování scanneru, a podle automatu byl scanner implementován v souboru `scanner.c`. Hlavičkový soubor `scanner.h` obsahuje datové typy, dokumentaci a deklaraci funkcí.

Datový typ **Token** obsahuje `TokenType` a `TokenValue`. **TokenType** je `enum` který definuje všechny typy tokenů: operační znaky, identifikátory, čísla, řetězce, klíčová slova, error token a další pomocné tokeny. **TokenValue** je `union` pro ukládání hodnot načtených identifikátorů, čísel, řetězců a booleanu.

Scanner nejdříve čeká, než parser zavolá `get_token()`, začne načítat znaky ze standardního vstupu a zpracovává je podle implementovaného automatu. Zpracované znaky si ukládá do **pole znaků** `buffer[]` a tam je podle potřeby dále upravuje. Například u escape sekvencí přepíše poslední načtené znaky odpovídajícím znakem escape sekvence.

Když scanner načte celý lexém, zavolá funkci `add_token()`, která u tokenů, které mají mít hodnotu, převede načtený řetězec uložený v poli na odpovídající hodnotu, zapíše ji do `TokenValue` daného tokenu a na konci už jen vrátí hotový token parseru. A pak zase čeká, než parser zavolá `get_token()`.

Klíčová slova jsou rozeznávána od identifikátorů přes **lookup tabulku**. Pokud scanner narazí na znak, který se na daném místě nesmí vyskytovat, vrátí token typu `ERROR`.

2.2 Syntaktická analýza

Syntaktický analyzátor (parser) představuje řídicí část překladače, který dohlíží na to, zda posloupnost tokenů získaná z lexikálního analyzátoru odpovídá gramatice jazyka IFJ25.

Samotný analyzátor je implementován v souboru `parser.c`.

V naší implementaci jsme k zpracování syntaxe použili dvě metody:

- **Metoda rekurzivního sestupu** pro analýzu hlavní kostry programu, definic, řídicích konstrukcí a funkcí. Tato část je řízena LL(1) gramatikou, její návrh se nachází na straně 11.
- **Precedenční syntaktická analýza**, která slouží ke zpracování výrazů v IFJ25.

Metoda rekurzivního sestupu je implementována pomocí sady funkcí, které odpovídají ne-terminálům naší LL gramatiky. Hlavní startovní bod je funkce `run_parser`, která nejprve ověří validitu prologu (pomocí funkce `valid`) a následně spustí analýzu těla programu zavoláním funkce `program`. Implementace rekurzivního sestupu se skládá z několika klíčových funkcí:

- `program`: Zpracovává hlavičku třídy `Program` a spouští analýzu hlavního bloku programu.
- `command`: Určuje typ příkazu na základě aktuálního tokenu (např. `VAR`, `ID`, `WHILE`, `IF`) a volá příslušnou podfunkci.
- `func_decl`: Zpracovává definice funkcí, getterů a setterů.
- `block`: Zpracovává sekvenci příkazů ve složených závorkách a ve smyčce volá funkci `command`, dokud nenarazí na uzavírací závorku.
- `cond_loop`: Funkce, která zpracovává podmínky (`if`) a cykly (`while`). Nejdříve zavolá precedenční analyzátor pro zpracování podmínky v závorkách a následně zpracuje tělo příkazu. Pro `if` ještě kontroluje přítomnost větve `else` a případně rekurzivně zpracovává řetězení `else if`.
- `assign`: Řeší pravidlo pro přiřazení hodnoty do proměnné. Kontroluje levou stranu (identifikátor) a pro pravou stranu volá precedenční analyzátor.
- `func_call` a `built_in_call`: Zpracovávají volání funkcí. `func_call` řeší uživatelské funkce a `built_in_call` zpracovává volání přes jmenný prostor `Ifj..` Obě funkce pro zpracování argumentů využívají pomocnou funkci `params`.
- `params` se používá při volání funkce a očekává seznam výrazů (skutečné argumenty), pro které se opakovaně volá precedenční analyzátor.
- `param_list` se používá při definici funkce a očekává seznam identifikátorů.
- `return_func`: Zpracovává příkaz `return` a ověřuje, zda za klíčovým slovem následuje výraz, a pokud ano, nechá jej zpracovat precedenčním analyzátozem.

Výsledkem syntaktické analýzy je **Abstraktní syntaktický strom** (n-ární strom), který ignoruje závorky, středníky či klíčová slova a představuje logickou strukturu programu. Tvorba tohoto stromu probíhá dvěma různými způsoby v závislosti na použité metodě analýzy. Konkrétní návrh stromu je blíže popsán v sekci 3.3. U rekurzivního sestupu při analýze řídicích konstrukcí (např. podmínky `if`, cykly `while...`) vzniká strom směrem od kořene k listům, kdy funkce odpovídající danému pravidlu (např. `cond_loop`) pro `if` nejdříve vytvoří rodičovský uzel (např. `NODE_IF`). Dále se pak volají jednotlivé funkce pro analýzu částí příkazu (např. pro podmínku `parse_expression` a `block` pro tělo). Tyto funkce pak vrací dané uzly, které se ihned připojí k rodičovskému uzlu, aby nedošlo ke ztrátě kontextu. V případě chyby v dané větvi je rodičovský uzel a celý jeho podstrom uvolněn, aby nedocházelo k úniku paměti.

2.2.1 Precedenční analýza ke zpracování výrazů

Pro syntaktickou analýzu výrazů, kde musíme dodržovat prioritu a asociativitu operátorů, jsme zvolili metodu precedenční analýzy pomocí precedenční tabulky 4. Tato část je implementována funkcí `parse_expression`, která řídí samotné zpracování, a pomocnou funkcí `do_reduction`, která potom provádí konstrukci abstraktního syntaktického stromu.

Jádrem analýzy je nekonečný cyklus ve funkci `parse_expression`. V každém kroku se porovnává nejvrchnější terminál na zásobníku tokenů s aktuálním tokenem na vstupu. Vyhodnocování se řídí **precedenční tabulkou** (`precedence_table`), která určuje priority mezi všemi operátory, závorkami a operandy.

Funkce `do_reduction` určí typ výrazu a zpracovává ho.

1. Ze zásobníku tokenů postupně odebírá symboly, dokud nenarazí na zarážku. Tím získá tzv. **handle** [1] (pravou stranu pravidla), např. `E + E` nebo `id`.
2. Podle délky handle a toho, jak handle vypadá, určíme, o jaký typ výrazu se jedná (např. binární operace, unární operace, operand).
3. Pro vytvoření odpovídajícího uzlu AST využívá **zásobník AST uzlů**. Pokud například redukuje pravidlo `E + E`, funkce odebere dva uzly z AST zásobníku (levý a pravý operand), vytvoří nový rodičovský uzel `NODE_BINOP` a ten vloží zpět na AST zásobník.
4. Na zásobník tokenů se místo zpracovaného handle vloží neterminál `E` (reprezentovaný pseudo-tokenem).

Funkce tento proces opakuje, dokud na vstupu i na zásobníku nezůstane jen ukončovací znak `$`, což znamená, že na AST zásobníku je kořen celého výrazu.

2.3 Sémantická analýza

Sémantická analýza je implementována v souboru `semantic.c`. K analýze se využívá již vygenerovaný AST od syntaktického analyzátoru a zajišťuje se správnost programu na základě významových pravidel. Syntaktický analyzátor je dvouprůchodový.

1. V první fázi (`PHASE_DEFINITION`) prochází v AST kořenový blok programu a znamenává všechny globální symboly (getter, setter, funkce) do globální tabulky symbolů. V této fázi se kontroluje pouze redefinice globálních symbolů.
2. V druhé fázi (`PHASE_ANALYSIS`) se procházejí samotná těla funkcí, kde následuje kontrola nedefinovaných proměnných, typové kompatibility výrazů a správy lokálních rozsahů platnosti.

Správa rozsahů a Name Mangling K vyřešení problému s viditelností proměnných jsme využili zásobník tabulek symbolů (`ScopeStack`). Dále, abychom vyřešili problém generátoru při překrývání proměnných se stejným názvem ve vnořených blocích (tzv. shadowing), využili jsme metodu zvanou "Name Mangling"[5], kdy jsou lokální proměnné v AST přejmenovány do formátu `funkce_hloubka_nazev` (např. `main_2_x`).

Transformace AST pro generátor Sémantická analýza kromě samotné kontroly chyb také upravuje AST pro jednodušší generování kódu:

- **Gettery:** Pokud je ve výrazu identifikován symbol jako getter, uzel `NODE_ID` je přepsán na `NODE_CALL` s nastaveným rámcem `FRAME_GLOBAL`. Generátor k němu přistupuje jako k volání funkce.
- **Settery:** Jsou v tabulce symbolů uloženy s klíčem obsahujícím znak `=` (např. `unicorn=`), čímž je zajištěna jejich jednoznačná identifikace při přiřazení.

2.4 Generátor cílového kódu

Pro implementaci generátoru kódu jsme zvolili přístup přímého generování IFJcode25 ze **sémanticky ověřeného AST**, jelikož poskytuje veškerá potřebná data pro úspěšné generování a je tedy jediným vstupem a zdrojem informací pro generátor kódu. Implementaci nalezneme v souboru `generator.c`.

Generování provádíme rekurzivním průchodem AST (ve většině případů formou post-order). Těchto průchodů je však několik; před generováním funkcí je třeba definovat všechny globální proměnné tak, aby byly dostupné v celém programu, a vedle hlavního rekurzivního průchodu AST se před generováním těla funkce (`NODE_FUNCTION`) na začátek generují definice lokálních proměnných bloku dané funkce.

Jazyk IFJ25 je dynamicky typovaný, proto bylo nutné zejména u binárních operací generovat ověření všech chybových stavů, které se mohou vyskytnout během běhu výsledného programu (nejčastěji chyby typů) a zajistit správné ukončení s chybou. Speciálně u aritmetických operací provádíme implicitní konverzi na datový typ `float` a rovněž vyhodnocení polymorfismu operátorů `+` a `*`.

U funkcí využíváme datový zásobník a naše konvence volání spočívá v přípravě argumentů na zásobník a vytvoření rámce volajícím před voláním funkce. Následně volaný uloží návratovou hodnotu na vrchol zásobníku a zároveň po sobě uklidí, díky čemuž je

řešení výrazů jednodušší. Vestavěné funkce jazyka IFJ25 generujeme přímo do toku instrukcí, což nám umožnilo přesnější kontrolu chybových stavů a zároveň snížilo režii oproti vytváření nových rámců a předávání argumentů do samostatných funkcí.

Řídící struktury programu vyžadovaly odlišení svých návěstí, aby se zabránilo kolizím, zejména ve vnořených blocích. Využíváme globální proměnnou `labelCounter`, pomocí které jsme dosáhli unikátnosti každého návěstí.

3 Použité datové struktury a algoritmy

Tato kapitola popisuje klíčové datové struktury, které jsou nezbytnou součástí našeho překladače a umožňují nám efektivní ukládání a zpracování informací o překládaném programu.

3.1 Tabulka symbolů

Pro ukládání informací o identifikátorech (proměnné, funkce) se využívá tabulka symbolů, implementovaná v souborech `syntable.c` a `syntable.h`. Na základě varianty zadání **TRP-izp** se jedná o hashovací tabulku s implicitním zřetězením (tzv. otevřená adresace). Při vkládání se vypočítá hash klíče a v případě kolize se hledá nejbližší volná pozice v poli ("linear probing"). Tabulka se dynamicky nealokuje při každém vkládání, ale má fixní velikost (zvolenou jako prvočíslo pro lepší distribuci hashe), což zajišťuje rychlý přístup k prvkům. Pro výpočet indexu v tabulce jsme implementovali hashovací funkci **PJW Hash** [4] (ve variantě používané pro formát ELF). Tato funkce využívá bitové posuny a operaci XOR k efektivnímu promíchání bitů vstupního řetězce, což je optimalizováno pro minimalizaci kolizí v tabulce symbolů. Díky kvalitní distribuci indexů se časová složitost operací vyhledávání a vkládání blíží v průměrném případě konstantnímu času $\mathcal{O}(1)$.

3.2 Zásobníky

V projektu využíváme několik typů zásobníků, které jsou implementovány jako dynamická pole s ukazatelem na vrchol (`topIndex`).

- **Zásobník tokenů a AST uzlů:** Využívá se v syntaktické analýze (soubor `parser.c`) pro potřeby precedenční analýzy výrazů.
- **Zásobník scopů (ScopeStack):** Implementován v `semantic.c`. Slouží k ukládání ukazatelů na lokální tabulky symbolů. Umožňuje sledovat zanořování bloků kódu. Při vstupu do bloku se na zásobník vloží nová tabulka, při výstupu se odebere, čímž je zajištěna správná viditelnost lokálních proměnných.

3.3 Abstraktní syntaktický strom (AST)

Vnitřní reprezentace programu je realizována pomocí n-árního stromu, definovaného v `ast.c` a `ast.h`. Základem je struktura `ASTNode`, která obsahuje:

- **Typ uzlu:** Výčtový typ (např. `NODE_IF`, `NODE_ID`).
- **Data:** Hodnota uzlu (např. název proměnné, hodnota literálu) a sémantické atributy (datový typ, rámec).
- **Potomci:** Dynamicky alokované pole ukazatelů na "děti". Díky tomu může mít každý uzel libovolný počet potomků, což je nezbytné pro reprezentaci složitých struktur, jako jsou bloky příkazů nebo seznamy argumentů.

4 Organizace týmu a práce

Na začátku samotného vývoje jsme absolvovali několik informativních schůzek, které sloužily pouze k diskuzím ohledně samotného zadání, jež měly zabránit chybným interpretacím jednotlivých implementačních požadavků v pozdějších fázích vývoje a zároveň objasnit danou teorii všem. Hned na začátku vývoje došlo k rozdělení práce a návrhu struktury projektu. Jako formu komunikace jsme využívali platformu **Discord** a také osobní schůzky, na kterých se každý týden probíral postup a případné problémy v projektu. Ke správě souborů jsme využívali systém **Git** společně se vzdáleným repozitářem hostovaným na serveru **GitHub**. Při implementaci jednotlivých částí překladače každý využíval vlastní větev, která byla po samotném testování připojena do hlavní větve. Díky tomu jsme mohli sledovat všechny změny v čase a vyvarovat se tak většímu množství konfliktů při společné práci na jednom souboru.

4.1 Rozdělení práce mezi členy

Tým se rozhodl, že každý člen dá 2% svého bodového podílu vedoucímu za organizaci, vedení týmu a spolupráci na více částech projektu.

Člen týmu	Přidělená práce
Patrik Lošťák	organizace práce, vedení týmu, kontrola nad vývojem, syntaktická analýza, sémantická analýza, testování, implementace rozšíření, dokumentace
Petr David Lanča	lexikální analýza, testování, dokumentace, prezentace
Šimon Čorej	syntaktická analýza, syntaktická analýza výrazů, testování, dokumentace
Sebastián Kuchta	generátor cílového kódu, testování, "project setup", dokumentace

Tabulka 1: Rozdělení práce v týmu mezi jednotlivými členy

5 Závěr

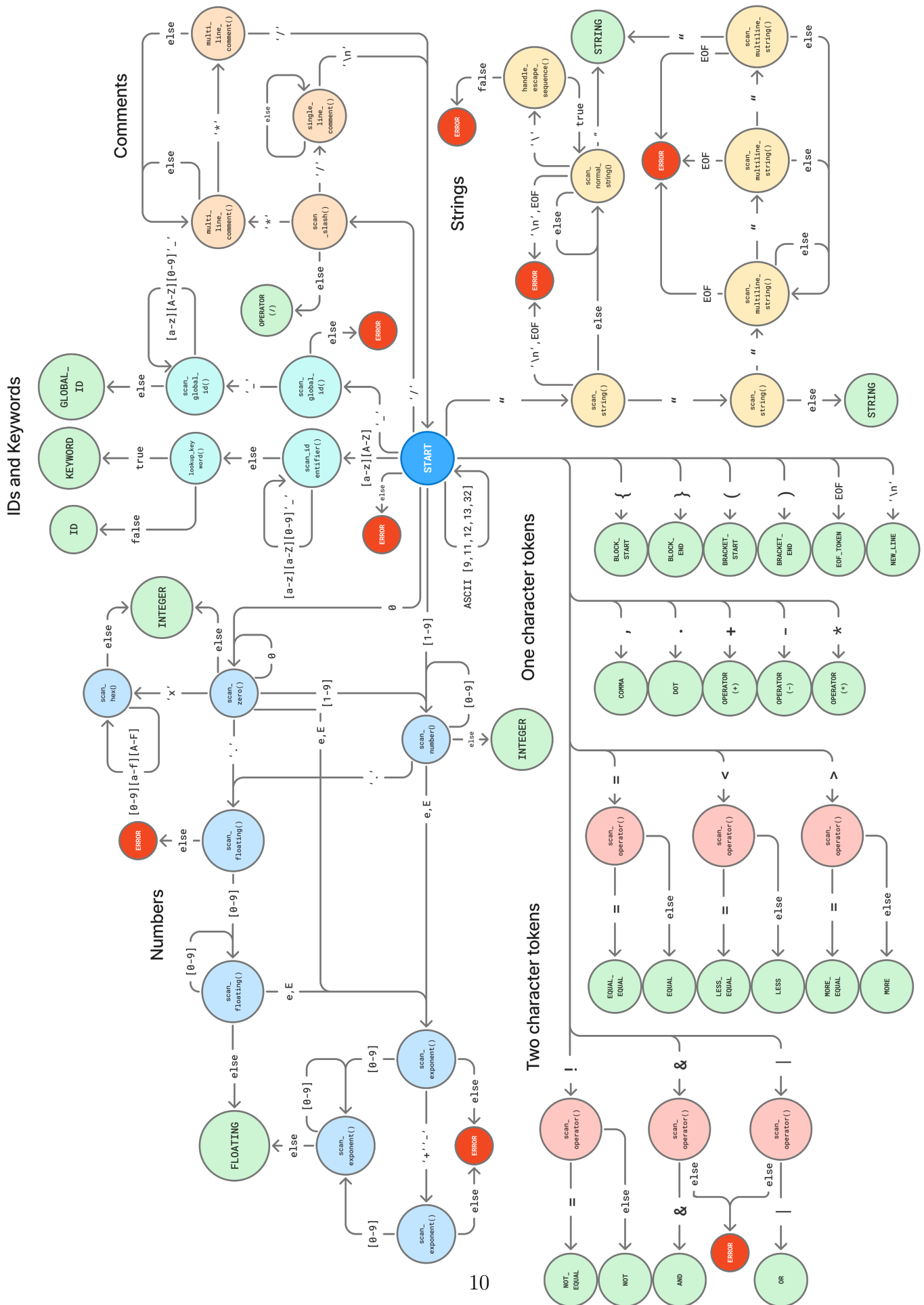
Jednalo se o první projekt většího rozsahu, který nám ukázal, že dobrý návrh a organizace práce na začátku vývoje vyřeší spoustu problémů. Největší chybou pro nás byla zpětná implementace tvorby AST do parseru, kdy muselo dojít k větší změně samotné struktury parseru. Dále prvotní návrh AST nedokázal rozlišit některé konstrukce, což se při implementaci sémantické analýzy a generátoru ukázalo jako velice problematické. Naštěstí samotná úprava návrhu nebyla náročná. V projektu jsme implementovali tři rozšíření (BOOLTHEN, OPERATORS, FUNEXP); první dvě jmenované jsme vyřešili dobrým návrhem precedenční analýzy, která pak i snadno vyřešila rozšíření FUNEXP.

Tým jako celek pracoval velmi dobře, všichni členové aktivně pracovali, a díky velmi dobré týmové komunikaci bylo možné odstranit všechny různé komplikace, které nastaly, v relativně krátkém časovém horizontu.

Literatura

- [1] GeeksforGeeks: Role of Operator Precedence Parser. [online], 2024, [vid. 2025-11-29]. Dostupné z: <https://www.geeksforgeeks.org/compiler-design-operator-precedence-parser/>
- [2] Křivka, Z.; Ondryáš, O.: *Zadání projektu z předmětů IFJ a IAL*. VUT FIT, Brno, 2025, rev. 21. září 2025.
- [3] Nystrom, B.: Wren Programming Language. [online], 2025, [vid. 2025-11-26]. Dostupné z: <https://wren.io/>
- [4] SSOJET: Compare Hashing Algorithms: Bernstein's hash vs PJW hash. [online], [vid. 2025-11-29]. Dostupné z: <https://ssojet.com/compare-hashing-algorithms/bernsteins-hash-djb2-vs-pjw-hash--elf-hash/>
- [5] Wikipedia: Name mangling. [online], rev. 2025, [vid. 2025-11-29]. Dostupné z: https://en.wikipedia.org/wiki/Name_mangling

Příloha A: Diagram konečného automatu



Příloha B: LL Gramatika

1. Program	-> Prolog class id l_curly Block_Body r_curly eof
2. Prolog	-> import string for ifj
3. Block_Body	-> eol Cmd_List
4. Cmd_List	-> Command Cmd_List
5. Cmd_List	-> eps
6. Command	-> static Func_Decl
7. Command	-> var id eol
8. Command	-> id Id_Suffix
9. Command	-> g_id assign Expr eol
10. Command	-> Built_In_Call
11. Command	-> return Return_Val
12. Command	-> if Cond_Loop
13. Command	-> while Cond_Loop
14. Command	-> l_curly Block_Body r_curly
15. Command	-> eol
16. Func_Decl	-> id Func_Body
17. Func_Body	-> l_par Params r_par l_curly Block_Body r_curly eol
18. Func_Body	-> assign l_par Params r_par l_curly Block_Body r_curly eol
19. Func_Body	-> l_curly Block_Body r_curly eol
20. Id_Suffix	-> assign Expr eol
21. Id_Suffix	-> l_par Args r_par eol
22. Built_In_Call	-> ifj dot id l_par Args r_par eol
23. Cond_Loop	-> l_par Expr r_par l_curly Block_Body r_curly Else_Part eol
24. Else_Part	-> else Else_Branch
25. Else_Part	-> eps
26. Else_Branch	-> if Cond_Loop
27. Else_Branch	-> l_curly Block_Body r_curly
28. Params	-> id Param_Next
29. Params	-> eps
30. Param_Next	-> comma id Param_Next
31. Param_Next	-> eps
32. Args	-> Expr Args_Next
33. Args	-> eps
34. Args_Next	-> comma Expr Args_Next
35. Args_Next	-> eps
36. Return_Val	-> Expr eol
37. Return_Val	-> eol

Tabulka 2: LL Gramatika použitá pro syntaktickou analýzu

Příloha C: LL a Precedenční tabulka

	import	class	static	var	id	g_id	ifj	return	if	while	{	}	eol	=	(	)	else	,	.	EXPR	\$
Program	1																				
Prolog	2																				
Block_Body													3								
Cmd_List			4	4	4	4	4	4	4	4	4	5	4								
Command			6	7	8	9	10	11	12	13	14		15								
Func_Decl					16																
Func_Body											19			18	17						
Id_Suffix														20	21						
Built_In_Call							22														
Cond_Loop															23						
Else_Part													25				24				
Else_Branch									26		27										
Params					28												29				
Param_Next																31		30			
Args																33				32	
Args_Next																35		34			
Return_Val													37							36	

Tabulka 3: LL Tabulka (Rozkladová tabulka)

	!	* /	+ -	< >	is	== !=	&&		()	?	:	id	type	\$
!	<	<	<	<	<	<	<	<	<	>	>	>	<	>
* /	>	>	>	>	>	>	>	>	<	>	>	>	<	>
+ -	<	<	>	>	>	>	>	>	<	>	>	>	<	>
< >	<	<	<	>	>	>	>	>	<	>	>	>	<	>
is	<	<	<	<	>	>	>	>	<	>	>	>	<	>
== !=	<	<	<	<	<	>	>	>	<	>	>	>	<	>
&&	<	<	<	<	<	<	>	>	<	>	>	>	<	>
	<	<	<	<	<	<	<	>	<	>	>	>	<	>
(	<	<	<	<	<	<	<	<	<	=	<	<	<	<
)	>	>	>	>	>	>	>	>	<	>	>	>	<	>
?	<	<	<	<	<	<	<	<	<	<	<	=	<	<
:	>	>	>	>	>	>	>	>	<	>	<	<	<	>
id	>	>	>	>	>	>	>	>	<	>	>	<	<	>
type	>	>	>	>	>	>	>	>	<	>	>	<	<	>
\$	<	<	<	<	<	<	<	<	<	<	<	<	<	<

Tabulka 4: Precedenční tabulka pro zpracování výrazů